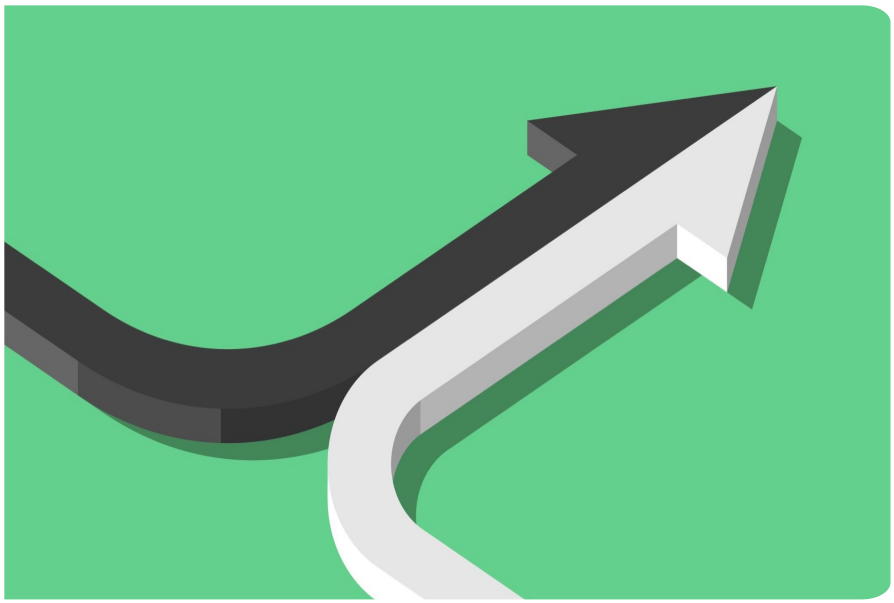


Object Oriented Programming - 5,6

The four pillars of object-oriented design :
Abstraction, Encapsulation, Inheritance, Polymorphism

Informatics Engineering, 2026

By the End of This Session You Will Be Able To :

1. **Design** the classes a system needs, before writing code
2. **Protect** an object's data so it can never become invalid
3. **Reuse** existing classes through inheritance, using extends and super
4. **Apply** the constructor rules of inheritance correctly
5. **Predict** which method runs when overloading or overriding applies
6. **Choose** between inheritance (is-a) and composition (has-a)

SOFTWARE : OBJECT ORIENTED

- Software that built from classes and objects that interact with each other.
  - Class

    A static description of something.
  - Object

    Something created (instantiation) of a class.

Difference between Class and Object

CLASS : the blueprint

- A description of something
- Declares the attributes and methods its objects will have
- Holds no real data by itself
- **Written once**

OBJECT : the instance

- A real thing created from a class with new
- Has its own independent data
- Can call the methods its class declares
- **Created many times**

KEY IDEA Class: Car.
Objects: Mr Joko's car, my car, the red car

same structure, independent data.

Feature of Object Oriented Programming

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

The Four Pillars of Object-Oriented Programming

Abstraction

What matters?

Model only what the problem needs; leave out the rest.

Encapsulation

Who may touch the data?

Hide internals; allow access only through methods.

Inheritance

How do I reuse?

Build a new class on top of an existing one.

Polymorphism

One call, many behaviours?

The same method call behaves differently per object.

KEY IDEA

Abstraction + Encapsulation = designing ONE class well.
Inheritance + Polymorphism = how classes RELATE.
Polymorphism needs inheritance.

OOP Feature 1 : Abstraction

What matters, and what can we leave out?

Abstraction (Design Level)

- Abstraction is a process of registering entities (objects) that will interact in an application.
- These entities will later become classes, and from classes are born objects.
- After the entities are registered, then the properties (state/attributes) and things that can be done by the entity (behaviour/method) are determined.

Two Levels of Abstraction

1. Design level

- A thinking activity, done before coding
- Decide which entities, attributes and behaviours the program needs
- Analogy: a map keeps the roads and leaves out the trees

2. Language level

- A Java feature: the abstract keyword
- A class states WHAT its subclasses must do, not HOW
- Cannot be instantiated with new

KEY IDEA Both levels share one idea: hide detail, so you can reason about what matters.

Design Abstraction, The Process

1

Define the domain

What system are we building? What is inside and outside its boundary?

2

List the candidate entities

Underline the NOUNS in the requirements, each is a candidate class.

3

Record attributes & behaviours

Attributes = data it holds (nouns). Behaviours = what it does (verbs).

4

Select only what you need

Drop irrelevant entities; merge duplicates. Your first list is always too long.

KEY IDEA Entities become classes → classes produce objects.

Case Study : Cooperative (Koperasi) Stock Application

The requirements

A cooperative wants an application that produces a WEEKLY STOCK REPORT, recording inventory, incoming goods and outgoing goods. A dedicated LOGISTICS OFFICER operates it, recording stock obtained from SUPPLIERS, both local and out of town. Goods are sold through a MINI-MARKET to BUYERS who may or may not be cooperative MEMBERS.

Business rules

- Food past its expiry date must be discarded; food with ≤ 6 months left gets a 50% discount (assume all food has a shelf life > 1 year).
- Any item whose stock is ≤ 40% of the required level must be restocked immediately.

⚠ **Your turn (2 min): underline the NOUNS. Which are candidate classes?**

Step 1 & 2: Candidate Entities (first draft)

Entity	Attributes	Operations
Item	ID, quantity, purchase price, selling price, expiry date, discount	calculate expiry, calculate selling price, calculate discount
Supplier	ID, name, location, item type	modify name, modify location
Cooperative	transaction list, warehouse	modify transaction list
Buyer	ID, name	modify ID, modify name
Warehouse	stock list, cooperative	modify stock, set cooperative
Transaction List	sales data [ ], purchase data [ ]	add sale, add purchase
Stock List	items [ ], item quantities [ ]	add item, remove item

KEY IDEA The business rules became methods. Attributes are nouns; operations are verbs. Seven classes is too many, so selection comes next.

Step 3 : Entity Selection Criteria

1. Relevance to the problem

- Does the entity exist within the boundary of the stated problem?
- Is it actually NEEDED to solve the problem?
- **Our problem is a weekly STOCK report, not supplier or customer management.**

2. Independent existence

- If it only makes sense as a property of something else → it is an ATTRIBUTE, not a class.
- If two entities always carry the same information → MERGE them.

Candidate	Kept?	Reason
Supplier	No	The report tracks stock, not suppliers
Buyer	No	No rule depends on who the buyer is
Warehouse	Merged	Its data is covered by Cooperative + Stock List

Result : The Selected Classes

Item
ID quantity purchasePrice sellingPrice expiryDate
calcExpiry() calcSellingPrice() calcDiscount()

Cooperative
stockList transactionList
modifyTransactionList()

TransactionList
salesData[ ] purchaseData[ ]
addSale() addPurchase()

StockList
items[ ] quantities[ ]
addItem() removeItem()

KEY IDEA "discount" was an attribute in the draft, now it is only calcDiscount(). It depends on today's date, so a stored value goes stale. Derivable values should be CALCULATED, not stored.

Four classes, their data and behaviour, and not one line of code written yet.

ABSTRACTION (Design Level)

1. Define the system/application domain
2. List the entities in the system/application → Classes
3. Record attributes and behaviours
4. Select only those entities (classes) that will definitely be used.

Detailed in APBO (Analisa Perancangan Berorientasi Object) subject.

OOP Feature 2 :

Encapsulation

Who is allowed to touch the data?

Encapsulation

- Encapsulation is a way of hiding the implementation details (in the form of attributes or methods) of a class.
- In OOP, the need for encapsulation arises from the sharing of data between methods.
- In programming, the principle of encapsulation is expressed by the identifiers private, default, protected, public and public.

What Is Encapsulation?

- **Bundling an object's data with the methods that operate on it, and HIDING the internal details.**
- Outside code does not change fields directly; it ASKS the object, through methods.
- The object stays in charge of its own data.

Analogy: the bank

You cannot walk into the vault and change your balance. A teller checks your request, then updates it.

Teller = method. Vault = private field.

Expressed in Java through four access levels:

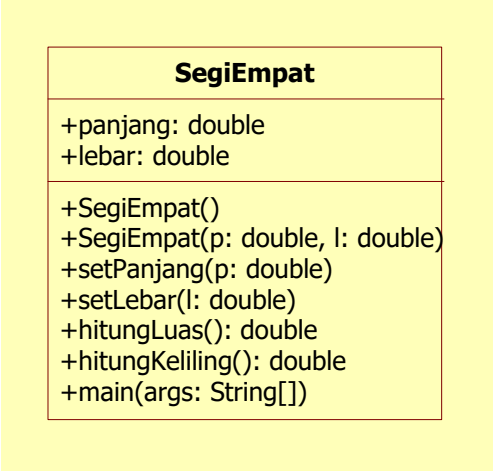
`private`	`default`	`protected`	`public`
class only	(no keyword) package	package + subclasses	everywhere

KEY IDEA The habit: fields private; public methods only for what outsiders genuinely need.

Basic Class Structure

- The basic structure of a class is essentially 4 parts, namely:
  - Class declaration;
  - Attribute declaration and initialisation;
  - Method definition;
  - Comments.

- UML (Unified Modeling Language) : Class Diagram



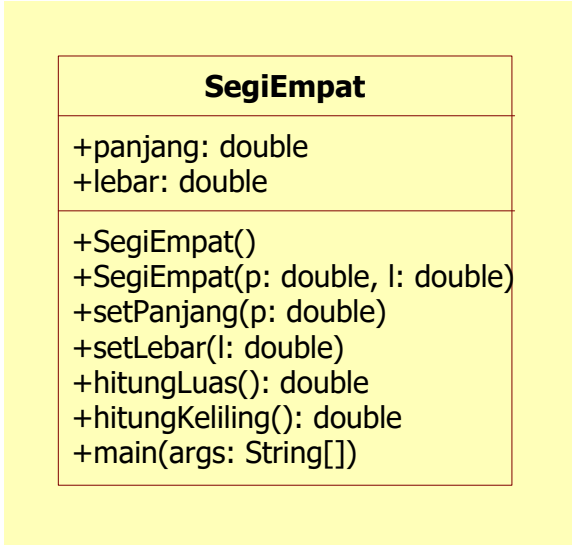
```
public class SegiEmpat{
    public double panjang;
    public double lebar;

    //deklarasi konstruktor
       public SegiEmpat(){
       }
       public SegiEmpat(double p, double l){
        panjang = p;
        lebar = l;
     }
     //deklarasi method
    public void setPanjang(double p){
           panjang = p;
    }
    public void setLebar(double l){
           lebar = l;
    }
    public double hitungLuas(){
           return panjang*lebar;
    }
    public double hitungKeliling(){
           return 2*(panjang + lebar);
    }

    public static void main(String[] args){
              SegiEmpat se1;
              se1 = new SegiEmpat(); //pembentukan objek
              se1.setPanjang(10);
              se1.setLebar(5);
              SegiEmpat se2 = new SegiEmpat(5.5,2.3); // pembentukan objek
              System.out.println("Luas segi empat 1 ="+se1.hitungLuas());
              System.out.println("Keliling segi empat 2 ="+se2.hitungKeliling());

        }
    }
```

Class Diagram With Code



The Problem : Public Fields

```
class Rectangle {
    public double length;     // anyone can change these
    public double width;

    public double area() {
        return length * width;
    }
}

public class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = -10;       // nothing stops this!
        r.width  = 5;
        System.out.println("Area = " + r.area());
    }
}
```

Output

```
Area = -50.0
```

What went wrong?

- It compiles.
- It runs.
- **It prints a confident, WRONG answer.**
- The class had no way to protect itself.

⚠ **Public fields let ANY code put your object into an invalid state, and no error tells you.**

The Fix : Private Fields + Validated Setters

```
class Rectangle {
    private double length;              // hidden
    private double width;

    public Rectangle() { this(1, 1); }  // valid default
    public Rectangle(double length, double width) {
        setLength(length);              // validation applies
        setWidth(width);                //   at creation too
    }
    public double getLength() { return length; }
    public void setLength(double length) {
        if (length > 0) this.length = length;
        else System.out.println("Rejected: length must be positive");
    }
    public void setWidth(double width) {
        if (width > 0) this.width = width;
        else System.out.println("Rejected: width must be positive");
    }
    public double area()      { return length * width; }
    public double perimeter() { return 2 * (length + width); }
}
```

```
Rectangle r1 = new Rectangle(10, 5);
Rectangle r2 = new Rectangle();
r2.setLength(5.5);
r2.setWidth(2.3);
r1.setLength(-10);    // rejected
print(r1.area());
print(r2.perimeter());
```

Output

```
Rejected: length must be positive
Area of r1      = 50.0
Perimeter of r2 = 15.6
```

KEY IDEA Constructor calls the setters,s so there is no back door.

Class Invariants : Why Encapsulation Really Matters

- **A class invariant is a rule about an object's data that is ALWAYS true, from creation onward.**

Class	Invariant
Rectangle	length > 0 and width > 0
BankAccount	balance ≥ 0
Date	1 ≤ month ≤ 12

```
// Invariant: length > 0 && width > 0
class Rectangle { ... }
```

With private fields

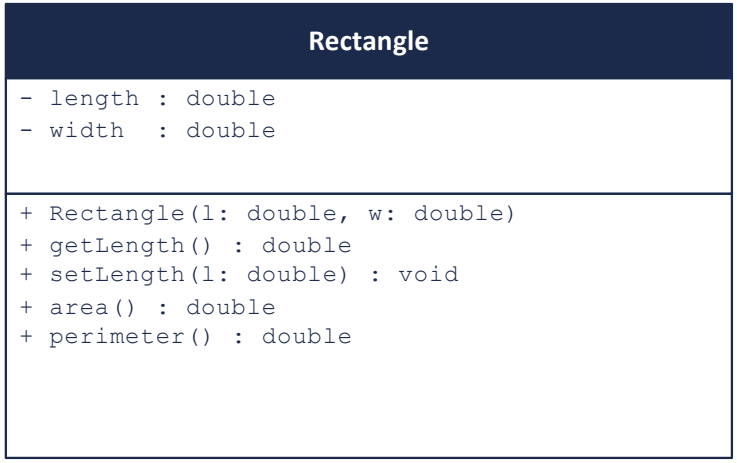
Only code INSIDE the class can break the invariant, so you only need to check the constructors and setters.

With public fields

ANY line in the whole program could break it.

KEY IDEA Encapsulation is not hiding for its own sake. It lets you make a PROMISE about your data, and keep it.

UML Class Diagrams : How We Draw a Class



← *class name*

← *attributes*

← *methods*

Symbol	Access level
–	private
~	default (package)
#	protected
+	public

KEY IDEA A class diagram shows the full interface of a class without a single line of implementation.

Access Modifiers : The Complete Picture

Modifier	Same class	Same package	Subclass (other package)	Everywhere else
`private`	✓	✗	✗	✗
`default (no keyword)`	✓	✓	✗	✗
`protected`	✓	✓	✓	✗
`public`	✓	✓	✓	✓

KEY IDEA Rule of thumb: start with private. Open up only for a specific reason, it is easy to widen access later, very hard to narrow it once others depend on it.

⚠ **Forget the keyword and you get DEFAULT (package) access, usually not what you intended.**

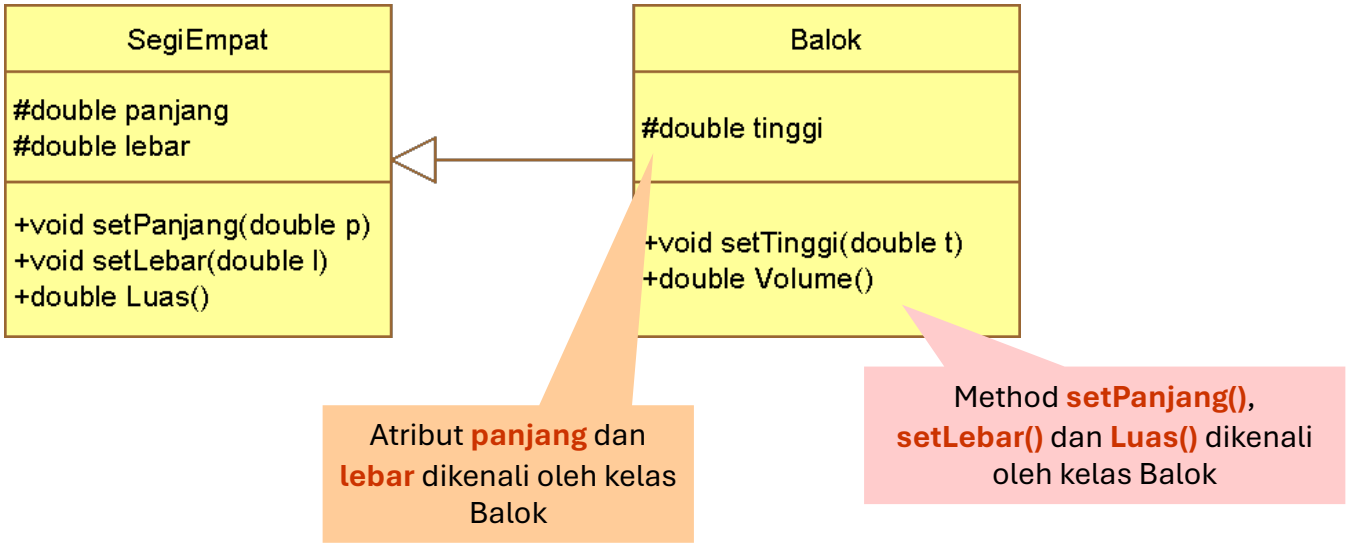
OOP Feature 3 :

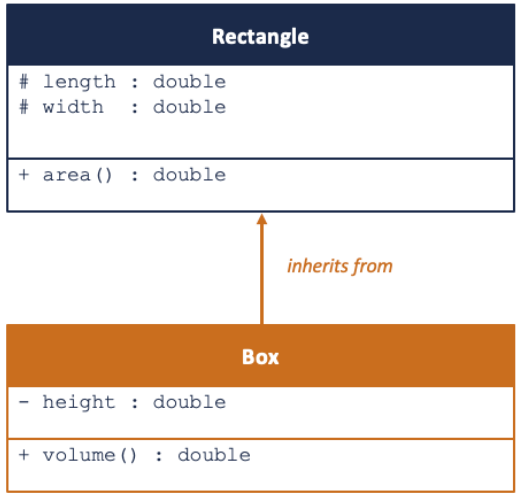
Inheritance

How do I reuse what already exists?

What Is Inheritance?

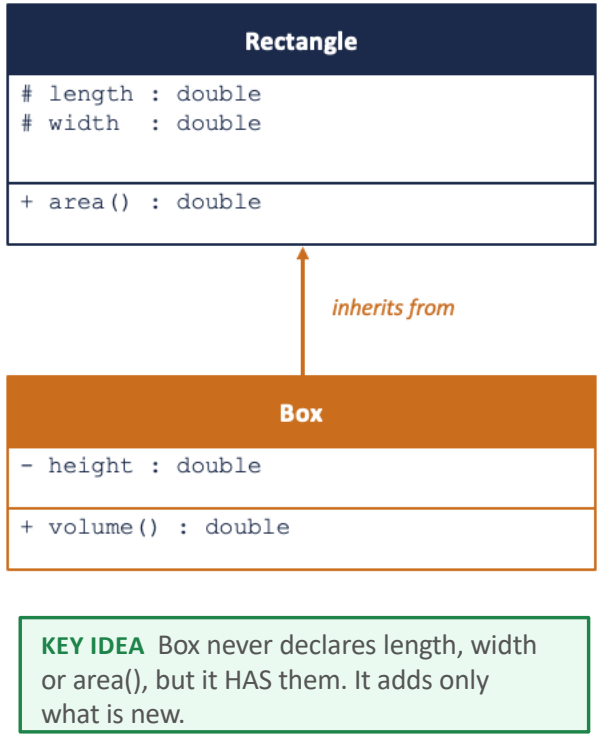
- The ability of a class to pass some or all of its attributes and methods to another class, so that those attributes and methods are recognised by the inheriting class without having to write them down.





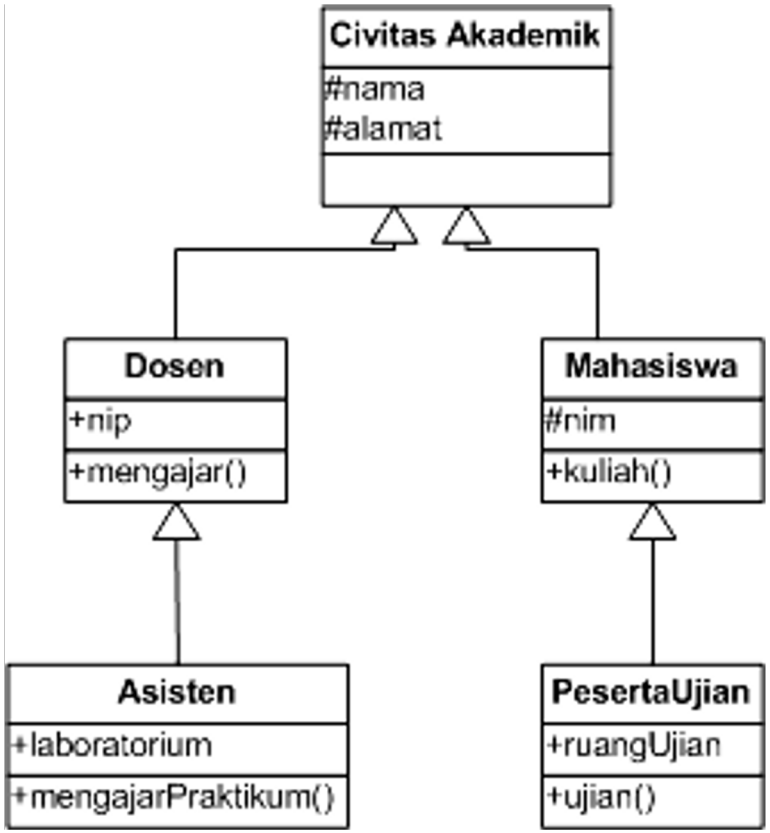
Inheritance ?

- The inheriting class is called the parent, super, or base class.
- The class that receives the inheritance is called the child class, derived class, or subclass.
- An implementation of the generalisation-specialisation or 'is-a' relationship between classes.



KEY IDEA Box never declares length, width or area(), but it HAS them. It adds only what is new.

EXAMPLE

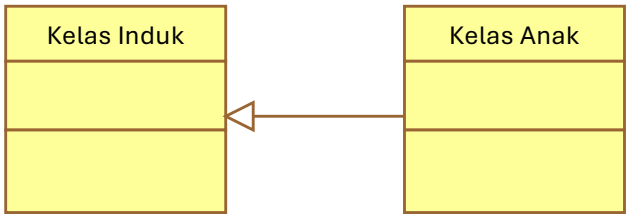


⚠ **The hollow arrow points from CHILD up to PARENT.**

Types of Inheritance

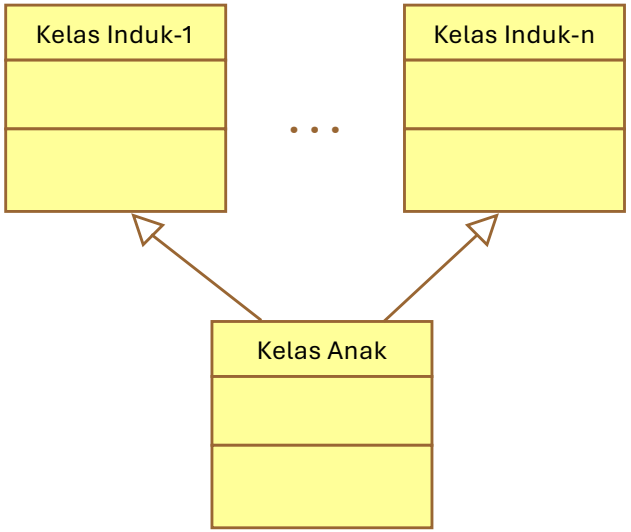
- *Single Inheritance*

  Child classes receive inheritance from one parent class.



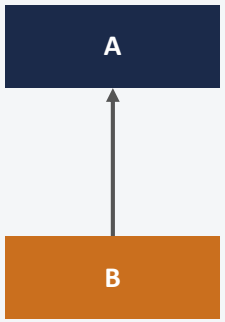
- *Multiple Inheritance*

  Child classes accept inheritance from multiple parent classes. The Java language does not accommodate multiple inheritance



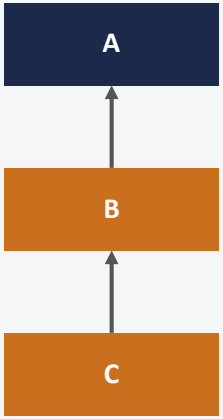
Types of Inheritance

Single



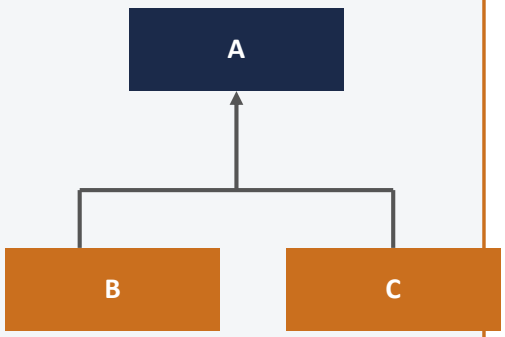
✓ Allowed

Multilevel



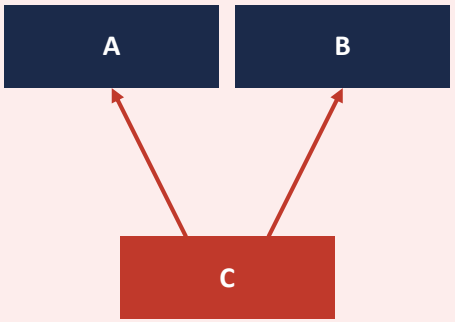
✓ Allowed

Hierarchical



✓ Allowed

Multiple ✗ in Java



✗ Not for classes

KEY IDEA Why no multiple inheritance? If both parents define the same method differently, which one would the child get? Java's answer is INTERFACES, a class may implement many (next week).

Inheritance in Java

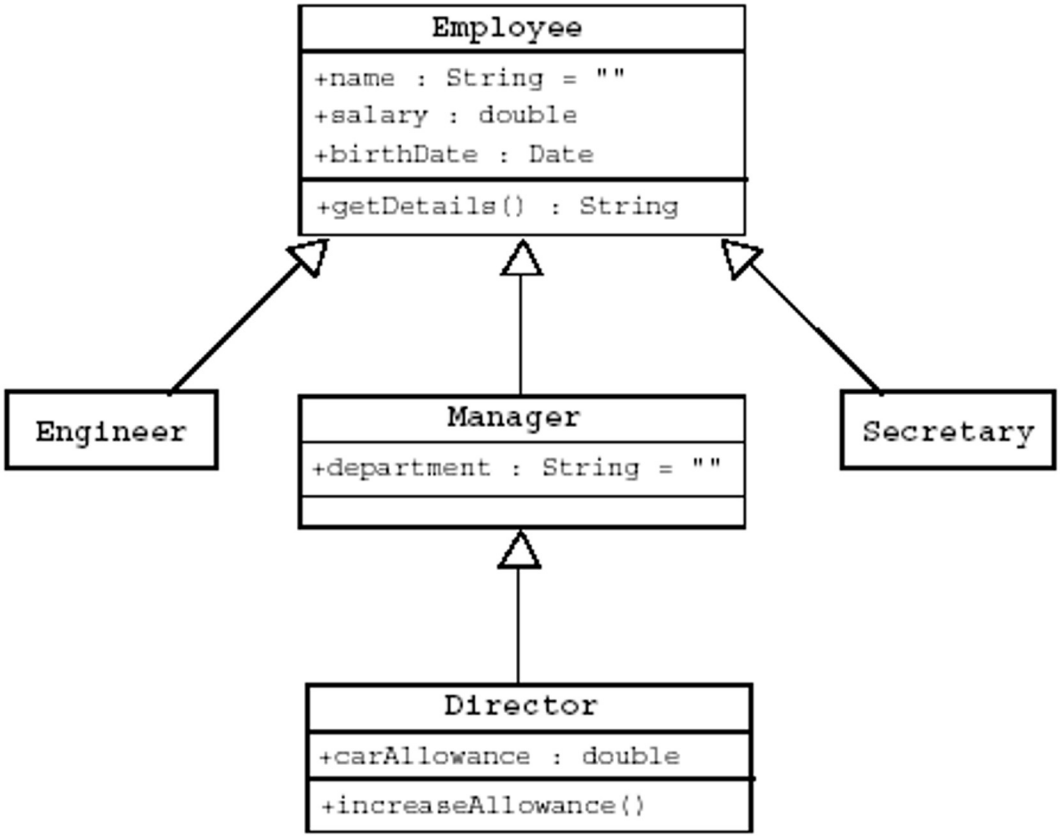
- Expressed using the extends keyword:

```
class KelasAnak extends KelasInduk {
  // Atribut kelas anak
  // Method kelas anak
}
```

- All attributes and methods with non-private access modifiers are inherited from the parent class to the child class.
- Inherited methods can be redefined in the child class (override) → polymorphism.

Inheritance in Java

- Single and multilevel inheritance



Inheritance in Java : extends

```
class ChildClass extends ParentClass {
    // only what the child ADDS or CHANGES
    // - new attributes
    // - new methods
    // - overridden methods
}
```

What the child receives

- public and protected members ✓
- default members, if in the same package ✓

What the child does NOT get directly

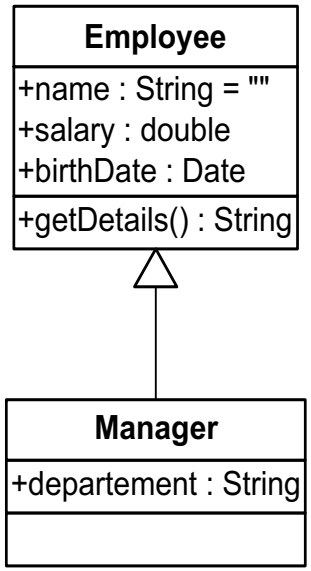
- private members : they exist in the object, but cannot be accessed by name
- **constructors : never inherited**

What the child can do

- Add new attributes and methods
- Redefine inherited methods : OVERRIDING → polymorphism

KEY IDEA Only single inheritance: a class may extend exactly ONE class.

Example 1

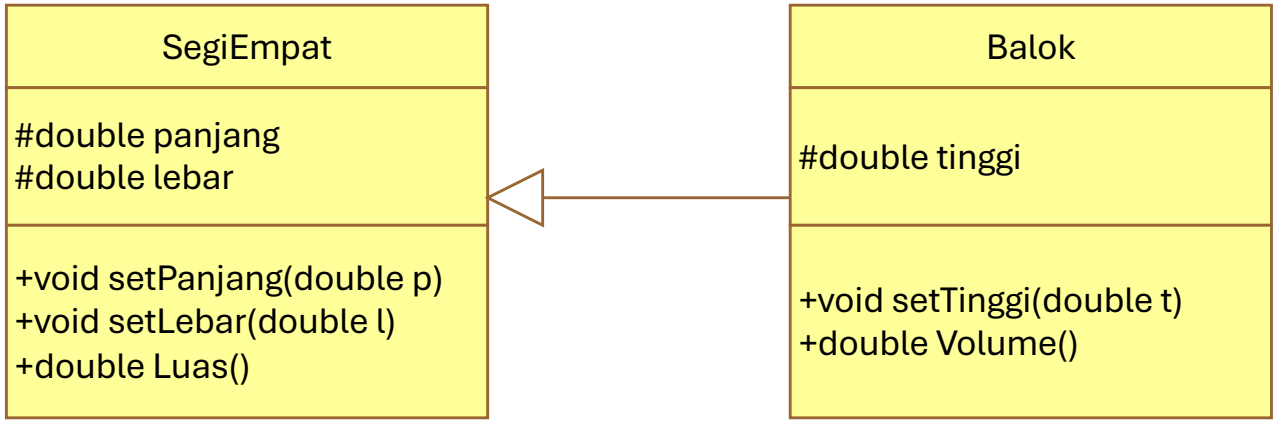



```
public class Employee {
  public String name;
  public Date birthDate;
  public double salary;

  public String getDetails() {...}
}

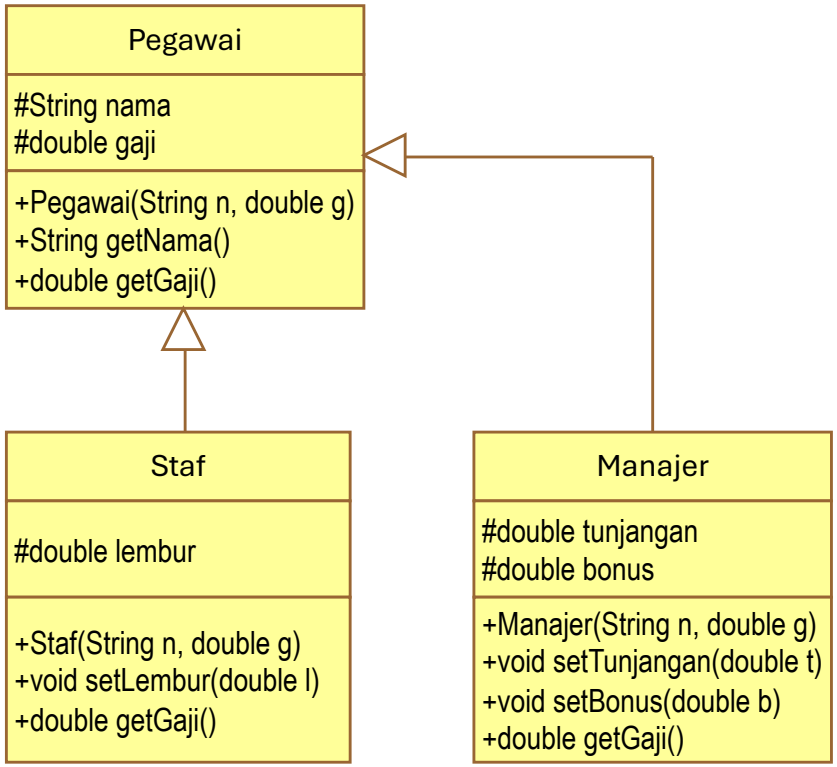
public class Manager extends
Employee {
  public String department;
}
```

Example 2



- SegiEmpat.java
- Balok.java
- Driver.java

Example



- Pegawai.java
- Staf.java
- Manajer.java
- Driver.java

Example 1 : Employee and Manager

```
public class Employee {
    public String    name;
    public LocalDate birthDate;
    public double    salary;
    public String getDetails() { ... }
}

public class Manager extends Employee {
    public String department;     // the ONLY new field
}
```

A Manager object holds:

```
name         ← inherited
birthDate    ← inherited
salary       ← inherited
getDetails() ← inherited
department   ← declared
```

4 fields, only 1 written.

KEY IDEA Change Employee once (e.g. add an ID) and Manager gets it automatically : no copied code to keep in sync.

⚠ **Public fields used here only to keep the first example short. Real code: private/protected.**

Example 2 : Bicycle and MountainBike

```
class Bicycle {
    private int gear;
    public void setGear(int increment) { gear = gear + increment; }
    public int  getGear()              { return gear; }
}

class MountainBike extends Bicycle {
    private int saddle;
    public void setSaddle(int amount) {
        saddle = getGear() - amount;   // via the getter!
    }
    public int getSaddle() { return saddle; }
}
```

```
MountainBike mb =
    new MountainBike();
mb.setGear(3);   // inherited
print(mb.getGear());
mb.setSaddle(1);
print(mb.getSaddle());
```

Output

```
3
2
```

KEY IDEA gear is private in Bicycle, so MountainBike must use getGear() : encapsulation holds even between parent and child.

When Should You Use Inheritance?

Code reduction

Several classes share the same attributes and methods : put them in a common parent.

Reusability

An existing class already does most of what you need.

Extension

New requirements are mostly covered by an existing class : extend it, add the difference.

But first, always apply the IS-A test

Use inheritance only if the child genuinely IS A kind of the parent, and should support EVERYTHING the parent supports.

KEY IDEA Inheritance is the STRONGEST coupling in Java: a change to the parent ripples into every child. Make it a deliberate choice.

is-a vs has-a : Inheritance vs Composition

INHERITANCE = is-a

A Manager IS AN Employee

class Manager
  extends Employee

COMPOSITION = has-a

A Customer HAS Accounts

class Customer {
 ArrayList<Account> accounts;

```
class Customer {                    // has-a
    private String name;
    private ArrayList<Account> accounts
        = new ArrayList<>();

    public void addAccount(Account a) {
        accounts.add(a);
    }
    public double totalBalance() {
        double sum = 0;
        for (Account a : accounts)
            sum += a.getBalance();
        return sum;
    }
}
```

Output

```
Ani total balance
= 750000.0
```

You built this last week!

KEY IDEA Test: say both sentences aloud.
"A Customer IS AN Account" ✗
"A Customer HAS AN Account" ✓.
Principle: FAVOUR COMPOSITION OVER INHERITANCE.use inheritance only for a genuine is-a.

Keyword `super`

- `super` used to reference the superclass of a class, which is to reference members of the superclass, both data attributes and methods.



Inheritance in Java

- To call the constructor of the parent class from the constructor of the child class, the notation super() is used and must be written on the first line of the constructor of the child class.

```
public Balok() {
   super();
   tinggi = 0;
}
```

- To call a method of the parent class from the child class, use the notation super.nameMethod().

```
public double getGaji() {
   double pokok = super.getGaji();
   return (pokok+lembur);
}
```

Keyword `super`



```
1.class Employee {
2.  public String name;

3.  public Employee (String s) {
4.       name = s
5.  }
6.}
```

```
1.class Manager extends Employee {
2.  private String alamat;
3.  public Manager(String nama, String s) {
4.       super.name = nama ;
5.       alamat = s;
6.  }
7.}
```

Example 3 : Rectangle and Box, with super

```
class Rectangle {
    protected double length;
    protected double width;
    public Rectangle(double length, double width) {
        this.length = length;
        this.width  = width;
    }
    public double area() { return length * width; }
}

class Box extends Rectangle {
    private double height;
    public Box(double length, double width, double height) {
        super(length, width);    // parent builds ITS part
        this.height = height;    // child builds ITS part
    }
    public double volume() { return area() * height; }  // reuse
}
```

```
Box b = new Box(4, 3, 2);
print(b.area());
print(b.volume());
```

Output

```
Base area = 12.0
Volume    = 24.0
```

KEY IDEA Each class initialises ONLY its own fields. super(...) must be the FIRST line.

Access Modifier

- In the real world, a parent entity may not pass on some of what it has to a child entity for some reason.
- This is also the case with the concept of inheritance in OOP.
- A parent class may not pass on some of its members to its subclasses.
- For example, we are trying to modify the Employee class.
- This is affected by the **access modifier.**

Example

```
public class Pegawai {
        private String nama;
        public double gaji;
}
```

- Try to compile the Manager class in the previous example.
- What happens?
- An error message will appear like this:

```
Manajer.java:5: nama has private access in Pegawai
                nama=n;
```

- This proves that the Manager class does not inherit the name member data from its parent class (Employee).

Example: protected

```
1. package adifferentpackage; // Class Ski now in
// a different package
2. Public class Ski {
3.     protected void applyWax() { . . . }
4. }
```

```
1. Public package sportinggoods;
2. class DownhillSki extends Ski {
3.     void tuneup() {
4.             applyWax(); → OK
5.             // other tuneup functionality here
6.     }
7. }
```

Constructor is not inherited

- The constructor of the parent class cannot be inherited to its subclasses.
- Consequently, every time we create a subclass, we must call the parent class constructor.
- Calling the parent constructor must be done on the first line of the subclass constructor.
- If we don't declare it explicitly, the Java compiler will add the parent class constructor call declaration in the subclass constructor.

Constructor is not inherited

- Before the subclass runs its own constructor, it will run the superclass constructor first.
- This happens because the subclass constructor implicitly adds a super()call to invoke the superclass constructor by the compiler.

Exercise:

```
public class Parent
{

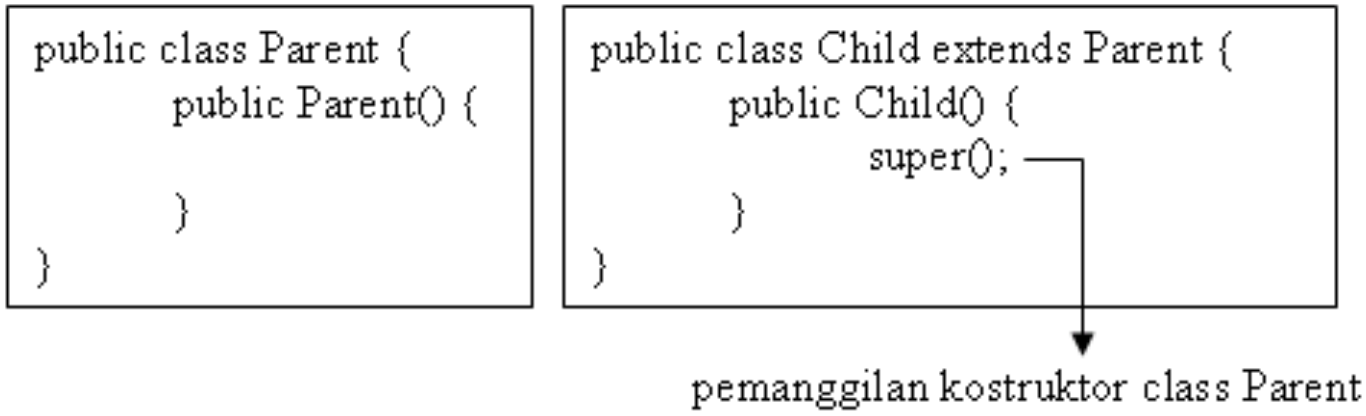
}
```

```
public class Child extends Parent {

}
```

- When the programme is compiled, the Java compiler will add a :
  - Parent class constructor
  - a Child class constructor
  - call the Parent class constructor in the Child class constructor

So the programme is the same as the following :



Example

```
public class Child extends Parent {
        int x;
        public Child() {
                x = 5;
                super();
        }
}
```

X

```
public class Child extends Parent {
        int x;
        public Child() {
                super();
                x = 5;
        }
}
```

√

Constructor

```
public class Person {
      protected String name;
      protected String address;

      public Person(){
            System.out.println("Inside Person:Constructor");
      }
}
```

```
public class Student extends Person {
      public Student(){
            System.out.println("Inside Student:Constructor");
      }
}
```

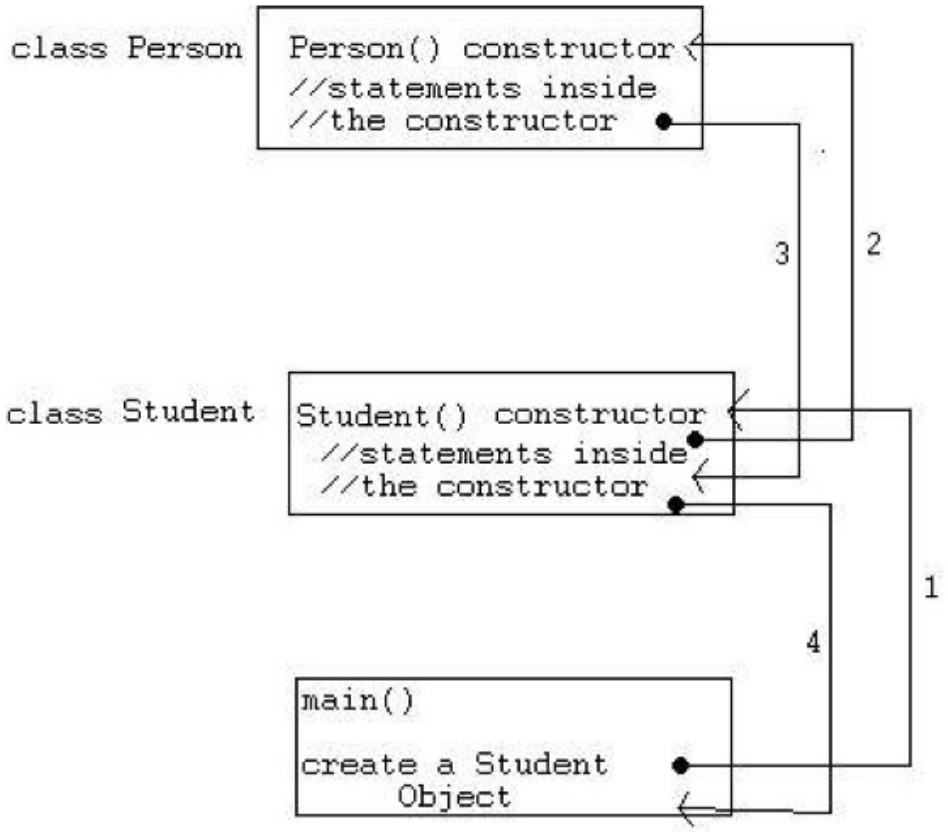
```
public static void main( String[] args ){
      Student rahmat = new Student();
}
```

- When a Student object is instantiated, the default constructor of the Student superclass is implicitly called to perform any necessary initialisation.
- After that, the statements in the constructor of the new subclass are executed.

The result:

```
Inside Person:Constructor
Inside Student:Constructor
```

Constructor Execution Flowchart



Superclass Constructor	Subclass Code	Compiler Action	Result
✅ Has **no-arg constructor** (default or explicit)	Subclass defines constructor without `super()`	Compiler inserts `super();`	✅ Compiles, superclass constructor runs first
✅ Has **parameterized constructor(s) only** (no no-arg)	Subclass defines constructor without `super(args)`	Compiler inserts `super();` but none exists	❌ Compile-time error
✅ Has **parameterized constructor(s)**	Subclass defines constructor and explicitly calls `super(args)`	Explicit `super(args)` matches superclass constructor	✅ Compiles, chosen parent constructor runs first
❌ Superclass has **no constructor at all**	(Java provides default no-arg constructor automatically)	Compiler inserts `super();`	✅ Compiles (default no-arg is created automatically)

Constructor Behavior in Java

If superclass has a **no-arg constructor**, you're safe (implicit super() works).
If superclass has **only parameterized constructors**, subclass must explicitly call one of them.
If superclass has **no constructor defined**, Java provides a hidden no-arg one.

1. Superclass with no-arg constructor

```
class Parent {
   Parent() { System.out.println("Parent no-arg"); }
}
class Child extends Parent {
   Child() { System.out.println("Child"); }
}
new Child();
```

Output:
Parent no-arg
Child

2. Superclass with only parameterized constructor (Error)

```
class Parent {
   Parent(String msg) { System.out.println(msg); }
}
class Child extends Parent {
   Child() { System.out.println("Child"); } // ❌ Error
}
```

Output:
Compile error: constructor Parent in class Parent cannot be applied to given types.

3. Parameterized constructor with explicit super(args)

```
class Parent {
   Parent(String msg) { System.out.println("Parent: " + msg); }
}
class Child extends Parent {
   Child() { super("Hello"); System.out.println("Child"); }
}
new Child();
```

Output:
Parent: Hello
Child

4. No constructor defined in superclass

```
class Parent { }  // compiler generates default no-arg constructor
class Child extends Parent {
   Child() { System.out.println("Child"); }
}
new Child();
```

Output:
Child

The Constructor-Chaining Trap

BROKEN

```
class Employee {
    public String name;
    public Employee(String s) { name = s; }
}
class Manager extends Employee {
    private String address;
    public Manager(String name, String s) {
        // compiler silently inserts super();
        super.name = name;
        address = s;
    }
}
```

Compiler says

```
error: constructor Employee in class Employee
  cannot be applied to given types;
  required: String
  found:    no arguments
```

FIXED

```
class Employee {
    public String name;
    public Employee(String name) { this.name = name; }
}
class Manager extends Employee {
    private String address;
    public Manager(String name, String address) {
        super(name);          // pass it up, FIRST line
        this.address = address;
    }
}
```

Output

```
Nia - Mataram
```

KEY IDEA If the parent has NO no-arg constructor, the child MUST call super(args).

Private Members and Inheritance

```
class Employee {
    private String name;
    public double salary;
    public Employee(String name) { this.name = name; }
    public String getName() { return name; }
}
class Manager extends Employee {
    public Manager(String n) {
        super(n);
        name = n;           // try to touch it directly
    }
}
```

Compiler says

```
error: name has private access in Employee
        name = n;
        ^
```

Both statements are TRUE

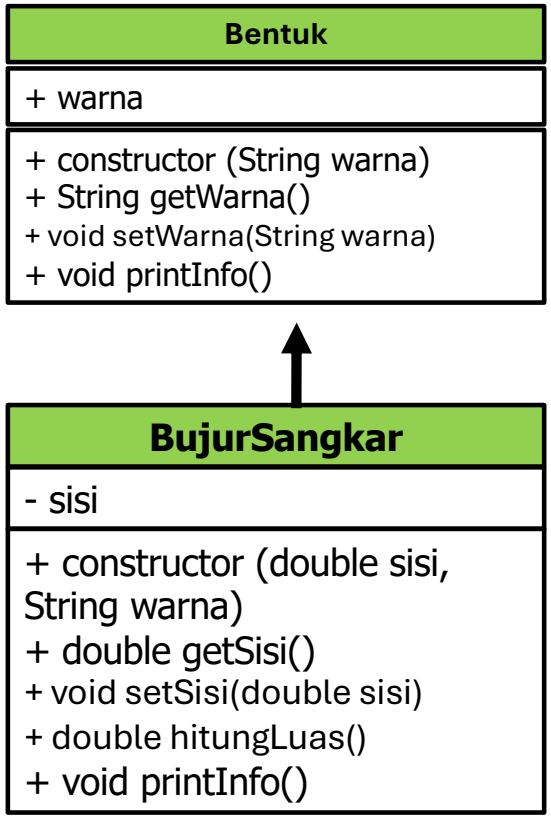
- Private members are NOT accessible by name in the subclass.
- The data STILL EXISTS inside every Manager object.

How Manager works with it

Through the parent's non-private methods: super(n) and getName().

KEY IDEA Present in the object, but not accessible to subclass code.

Exercise 1



Untuk kelas Bentuk :

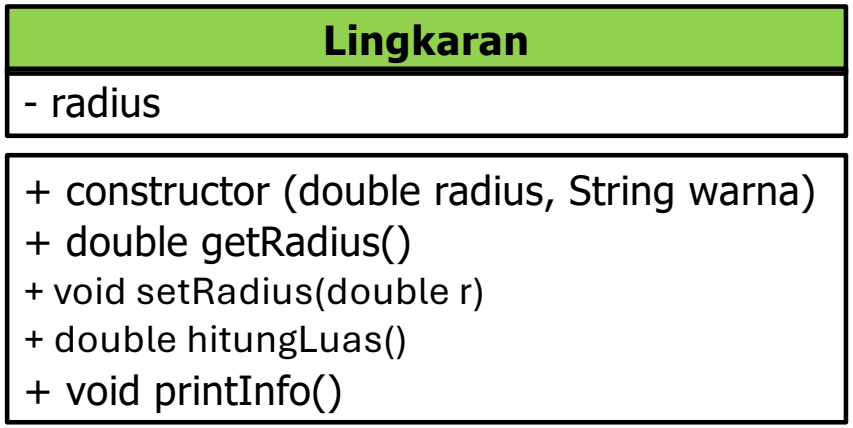
- **getWarna** adl method yang akan mengembalikan nilai variabel warna
- **setWarna** adl method untuk mengubah nilai variabel warna
- **printInfo** adl method yang akan menuliskan "Bentuk berwarna [warna]"

Untuk kelas BujurSangkar :

- **getSisi** adl method yang akan mengembalikan nilai variabel sisi
- **setSisi** adl method untuk mengubah nilai variabel sisi
- **hitungLuas** adl method yang akan mengembalikan hasil perhitungan luas bujursangkar
- **printInfo** adl method yang akan menuliskan "Bujursangkar berwarna [warna], luas = [luas]"

Exercise 2

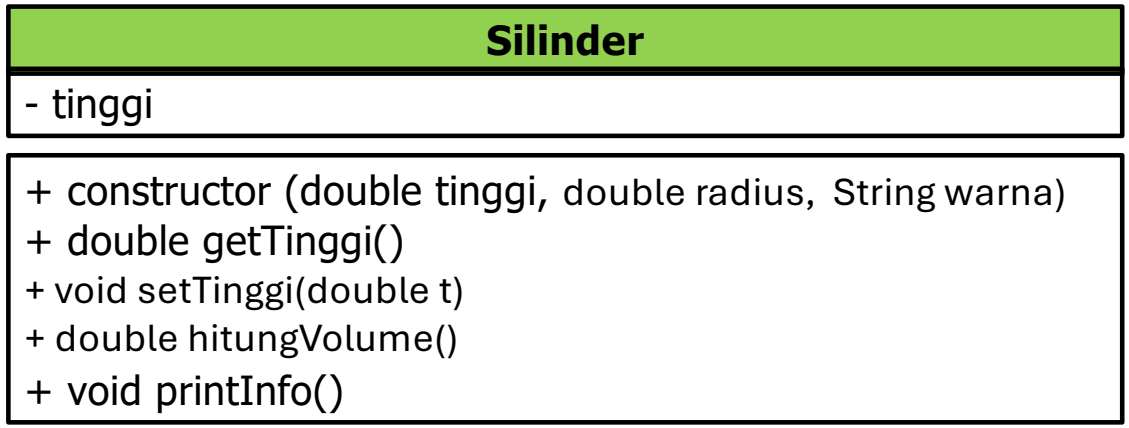
- Buatlah kelas Lingkaran sbg turunan kelas Bentuk.



- **constructor** akan menginisialisasi radius dan warna
- **getRadius** adl method yang akan mengembalikan nilai variabel radius
- **setRadius** adl method untuk mengubah nilai variabel radius
- **hitungLuas** adl method yang akan mengembalikan hasil perhitungan luas lingkaran (PHI jadikan sbg konstanta kelas)
- **printInfo** adl method yang akan menuliskan “Lingkaran [warna], luas = [luas]”

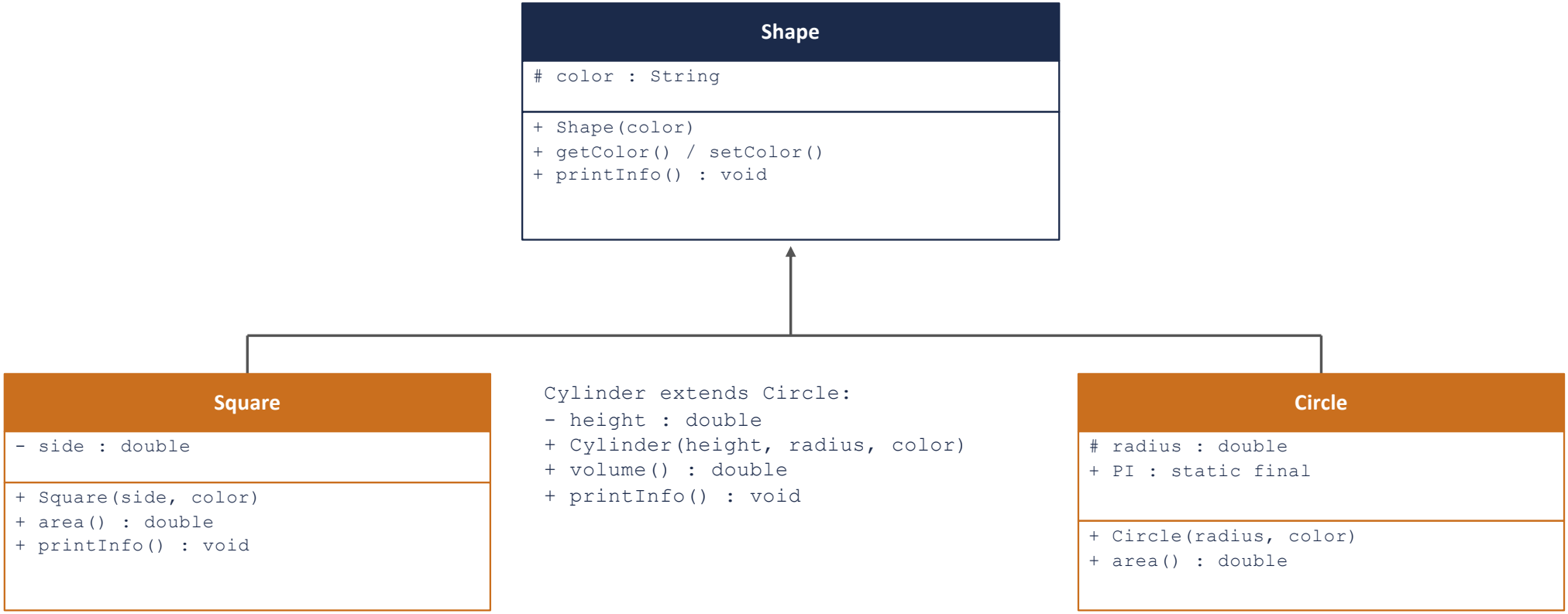
Exercise 3

- Buatlah kelas Silinder sbg turunan kelas Lingkaran



- **constructor** akan menginisialisasi variabel tinggi, radius, dan warna
- **getTinggi** adl method yg akan mengembalikan tinggi
- **setTinggi** adl method yg akan mengubah tinggi
- **hitungVolume** adl method yg akan mengembalikan hasil perhitungan volum silinder
- **printInfo** adl method yg akan menuliskan “Silinder warna [warna], volume = [volume]”

Exercise : Shape, Square, Circle, Cylinder (English Version of exercise 1,2,3)



Required output format, at printInfo() :

```
"Square colored [color], area = [area]"
"Circle [color], area = [area]"
"Cylinder [color], volume = [volume]"
```

OOP Feature 4 :

Polymorphism

One call, many behaviours?

Polymorphism

- *Polymorphism allows a method to have different forms*
- *There are 2 types of polymorphism:*
  - *Overloading*
  - *Override*



Polymorphism

- Ability to deal with objects that have different behaviours (forms)
- Introduction of the concept of polymorphism:
  - Overloading: The ability to use the same name for multiple methods that have different parameters (type and/or number).
  - Overriding: The ability of a subclass to override a method of a superclass by using the same name and parameters in the method.

What Is Polymorphism?

poly = many **morph** = form

The same method call behaves differently depending on the context.

COMPILE-TIME polymorphism

- **Achieved by OVERLOADING**
- Same name, different parameter lists
- The COMPILER picks the method from the arguments
- Also called static / early binding

RUN-TIME polymorphism

- **Achieved by OVERRIDING**
- A subclass redefines an inherited method
- The ACTUAL OBJECT decides, while running
- Also called dynamic / late binding

⚠ **Static binding and dynamic binding are the two OPPOSITE mechanisms behind the two kinds, not synonyms for polymorphism.**

Overloading Methods : Same Name, Different Parameters

```
class Calculator {
    public int    add(int a, int b)                 { return a + b; }
    public int    add(int a, int b, int c)          { return a + b + c; }
    public double add(double a, double b)           { return a + b; }
    public double add(double a, double b, double c) { return a + b + c; }
}

Calculator c = new Calculator();
print(c.add(23, 34));             // (int, int)
print(c.add(12, 28, 14));         // (int, int, int)
print(c.add(3.4, 4.9));           // (double, double)
print(c.add(12.5, 28.7, 14.2));   // (double, double, double)
```

Output

```
57
54
8.3
55.400000000000006
```

Must differ in…

- number of parameters, or
- their types, or
- their order

KEY IDEA 55.400000000000006 is NOT a bug: 12.5, 28.7 cannot be stored exactly in binary.

Polymorphism – Overloading

```
class Mobil {
    String warna;
    int tahunProduksi;

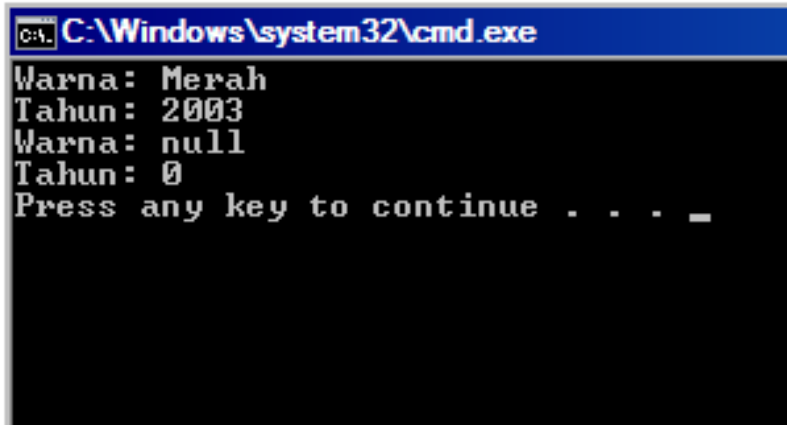
    public Mobil(String warna, int tahunProduksi){
       this.warna = warna;
       this.tahunProduksi = tahunProduksi;
    }

    public Mobil(){
    }

    void info(){
       System.out.println("Warna: " + warna);
       System.out.println("Tahun: " + tahunProduksi);
    }
}
```

```
public class MobilKonstruktor{
  public static void main(String[] args){
     Mobil mobilku = new Mobil("Merah", 2003);
     mobilku.info();

     Mobil mobilmu = new Mobil();
     mobilmu.info();
     }
}
```



Polymorphism : Overloading

```
class Lingkaran{
    void gambarLingkaran(){
    }
     void gambarLingkaran(int diameter){
    ...
    }
     void gambarLingkaran(double diameter){
    ...
    }
     void gambarLingkaran(int diameter, int x, int y){
    ...
    }
     void gambarLingkaran(int diameter, int x, int y, int warna, String
    namaLingkaran){
    ...
    }
}
```

Polymorphism - Overriding

```
public class Sepeda{
	private int gir;

		void setGir(int pertambahanGir) {
			gir= gir+ pertambahanGir;
		}

		int getGir() {
			return gir;
		}
}
```

Polymorphism - Overriding

```
public class SepedaGunung extends
   Sepeda{

   void setGir(int pertambahanGir) {
        super.setGir(pertambahanGir);
         gir = 2*getGir();
   }
}
```

SepedaGunung.java

```
public class SepedaGunungBeraksi {
  public static void main(String[] args) {

        SepedaGunung sg=new SepedaGunung();

        sg.setGir(2);
        System.out.println(sg.getGir());

        sg.setGir(3);
        System.out.println(sg.getGir());
      }
}
```

SepedaGunungBeraksi.java

Exercise : Overloading

1. Kembangkan class Matematika, MatematikaCanggih dan MatematikaBeraksi
2. Lakukan overloading pada Method yang ada (pertambahan, pengurangan, perkalian, pembagian, modulus)
3. Tambahkan method baru bertipe data double (pecahan) dan memiliki 3 parameter
4. Uji di kelas MatematikaBeraksi dengan parameter pecahan: 12.5, 28.7, 14.2
5. Uji konsep overloading dengan:
   pertambahan(12.5, 28.7, 14.2)
   pertambahan(12, 28, 14)
   pertambahan(23, 34)
   pertambahan(3.4, 4.9)

Overloading Constructor

```
1. public class Employee {
2.           private static final double BASE_SALARY = 15000.00;
3.           private String name;
4.           private double salary;
5.           private Date birthdate;

6.           public Employee (String name, double salary, Date DoB) {
7.                       this.name = name;
8.                       this.salary = salary;
9.                       birthdate = DoB;
10.          }
11.          public Employee(String name, double salary) {
12.                      this(name,salary,null);
13.          }
14.          public Employee(String name, Date DoB)  {
15.                      this(name,BASE_SALARY,DoB);
16.          }
17.          public Employee(String name)            {
18.                      this(name,BASE_SALARY);
19.          }
20.          //more Employee code...
21.}
```

Overloading Method

```
1. public class Employee {
2.           private static final double BASE_SALARY = 15000.00;
3.           private String name;
4.           private double salary;
5.           private Date birthdate;

6.           public Employee (String name, double salary, Date DoB) {
7.                       this.name = name;
8.                       this.salary = salary;
9.                       birthdate = DoB;
10.          }
11.          public void set_Employee(String name, double salary) {
12.                      this(name,salary,null);
13.          }
14.          public void set_Employee(String name, Date DoB)      {
15.                      this(name,BASE_SALARY,DoB);
16.          }
17.          public void set_Employee(String name)   {
18.                      this(name,BASE_SALARY);
19.          }
20.          //more Employee code...
21.}
```

Overriding Methods



- A subclass can redefine a method inherited from the parent class.
- A subclass can define a method that is functionally different from the method inherited from the parent class, as long as it has similarities:
  - Method name
  - Return type
  - Argument list

Overriding : The Rules

Rule	Detail
Same name	Must match exactly
Same parameter list	Different parameters = OVERLOADING, not overriding
Same return type	…or a subtype of it (covariant return, Java 5+)
Cannot reduce visibility	public in parent → must stay public. May widen, never narrow.
Cannot override	private, static, or final methods

```
class A { public void show() { } }
class B extends A {
    @Override
    void show() { }       // default < public
```

Compiler says

```
error: show() in B cannot override
  show() in A
  attempting to assign weaker access
  privileges; was public
```

Overriding Methods



```
1. class Employee {
2.   private String name;
3.   private MyDate birthDate;
4.   private float  salary;

5.   public String getDetails() {
6.     return "Name: " + name + "\nSalary: " + salary
7.            + "\nBirth Date: " + birthDate;
8.   }
9. }
```

```
1. class Manager extends Employee {
2.   protected String department;

3.   public String getDetails() {
4.     return super.getDetails() + "\nDepartment: " +
5.     department;
6.   }
7. }
```

Inheritance

```java
public class Kendaraan {
    private String nama_pemilik;
    protected String nomor_mesin;
    public int harga;

    public Kendaraan(){
        nama_pemilik = "Negara";
        nomor_mesin = "0";
        harga = 0;
        System.out.println("Ctor Kendaraan");
    }

    public Kendaraan(String pemilik, String mesin, int price){
        nama_pemilik = pemilik;
        nomor_mesin = mesin;
        harga = price;
        System.out.println("Copy Ctor Kendaraan");
    }

    public void deskripsi(){
        System.out.println("Kendaraan ini milik " + nama_pemilik);
        System.out.println("Nomor Mesin kendaraan ini " + nomor_mesin);
        System.out.println("Harga kendaraan ini = " + harga);
    }

    public void identitas(){
        System.out.println("Ini kendaraan");
    }
}
```

```java
public class Motor extends Kendaraan{
    public Motor(){
        System.out.println("Ctor Motor");
        System.out.println("Harga Motor = " + harga);// bisa
        System.out.println("Nomor Mesin Motor = " + nomor_mesin);// bisa
        //System.out.println("Pemilik Motor = " + nama_pemilik);// TIDAK bisa
    }
}
```

```java
public static void main(String[] args) {
    // TODO code application logic here
    Motor m1 = new Motor();
    m1.deskripsi();
}
```

```
Output - Konsep_PBO (run)
run:
Ctor Kendaraan
Ctor Motor
Harga Motor = 0
Nomor Mesin Motor = 0
Kendaraan ini milik Negara
Nomor Mesin kendaraan ini 0
Harga kendaraan ini = 0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Inheritance

```
11  public class Motor extends Kendaraan{
12      public Motor(){
13          System.out.println("Ctor Motor");
14          System.out.println("Harga Motor = " + harga);// bisa
15          System.out.println("Nomor Mesin Motor = " + nomor_mesin);// bisa
16          //System.out.println("Pemilik Motor = " + nama_pemilik);// TIDAK bisa
17      }
18  }
```

```
17      public static void main(String[] args) {
18          // TODO code application logic here
19          Motor m1 = new Motor();
20          m1.deskripsi();
21      }
```

```
Output - Konsep_PBO (run)
run:
Ctor Kendaraan
Ctor Motor
Harga Motor = 0
Nomor Mesin Motor = 0
Kendaraan ini milik Negara
Nomor Mesin kendaraan ini 0
Harga kendaraan ini = 0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Polymorphism

```
public class Kendaraan {
    private String nama_pemilik;
    protected String nomor_mesin;
    public int harga;

    public Kendaraan(){...}

    public Kendaraan(String pemilik, String mesin, int price){...}

    public void deskripsi(){...}

    public void identitas(){
        System.out.println("Ini kendaraan");
    }
}
```

```
public class Motor extends Kendaraan{
    public Motor(){...}

    @Override
    public void identitas(){
        System.out.println("Ini Motor");
    }
}
```

```
public class Mobil extends Kendaraan {

    @Override
    public void identitas(){
        System.out.println("Ini Mobil");
    }
}
```

Polymorphism

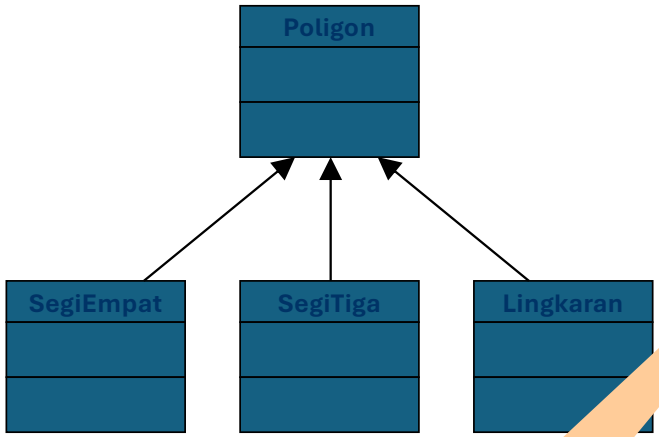
```
17      public static void main(String[] args) {
18          // TODO code application logic here
19          Kendaraan k = new Kendaraan();
20          Kendaraan mtr = new Motor();
21          Kendaraan mbl = new Mobil();
22
23          k.identitas();
24          mtr.identitas();
25          mbl.identitas();
26      }
```

```
Output - Konsep_PBO (run)
run:
Ctor Kendaraan
Ctor Kendaraan
Ctor Motor
Harga Motor = 0
Nomor Mesin Motor = 0
Ctor Kendaraan
Ini kendaraan
Ini Motor
Ini Mobil
BUILD SUCCESSFUL (total time: 0 seconds)
```

What is Polymorphism?

- The ability of an object to be used in many different targets with the same name.
- The ability of an object to respond differently to messages with the same name.
- Also known as
  - dynamic binding
  - static binding
  - runtime binding

Polimorphism :




```
// Program Utama
class TesPoligon {
  Poligon p = new Poligon();
  ...

  SegiEmpat s = new SegiEmpat();
  SegiTiga  t = new SegiTiga();
  Lingkaran l = new Lingkaran();

  p = s; // ok krn instan kelas
  anak juga instan kelas induk
  ...
}
```

variabel **p** menunjuk **objek** yang diciptakan dari kelas **Poligon**

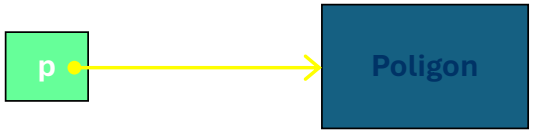
variabel **p** diubah sehingga menunjuk **objek** dari kelas **SegiEmpat**

variabel **s** menunjuk **objek** yang diciptakan dari kelas **SegiEmpat**

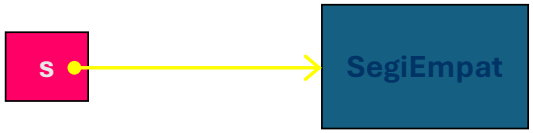
Polymorphism

Before:

Poligon p = new Poligon();

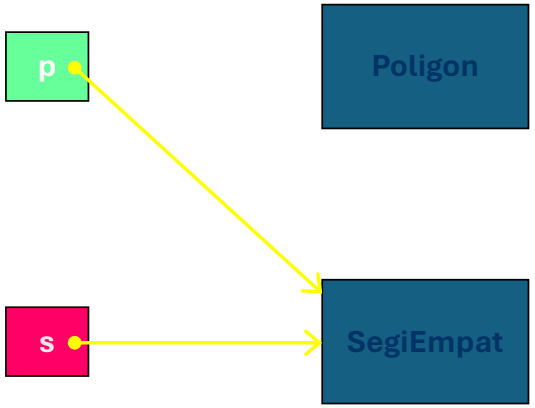


SegiEmpat s = new SegiEmpat();



After:

p = s;



Polymorphism Overview

```
class SegiEmpat {
  . . .
  public String display() {
    return ("Segi Empat"
     + "\no Panjang  : " + getPanjang()
     + "\no Lebar    : " + getLebar()
     + "\no Luas     : " + Luas()
     + "\no Keliling : " + Keliling());
  }
}
```

```
class Lingkaran {
  . . .
  public String display() {
    return ("Lingkaran"
     + "\no jari-jari: " + getRadius()
     + "\no Luas     : " + Luas()
     + "\no Keliling : " + Keliling());
  }
}
```

```
class TesPoligon {
  public static void main(String args[])
  {
    // Deklarasi array
    Poligon[] p = new Poligon[2];

    // Add array poligon
    p[0] = new SegiEmpat(17, 8);
    p[1] = new Lingkaran(10);

    // Display informasi
    for (int i=0; i<p.length; i++) {
      System.out.println(p[i].display());
    }
  }
}
```

Method **display ()** yang dieksekusi ditentukan berdasarkan rujukan ke masing-masing objeknya

Polymorphism

- An array that contains different geometry shapes

Kelas induk:

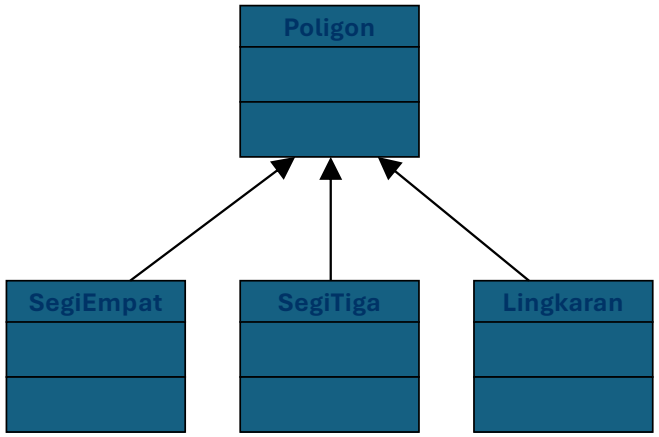
- Poligon.Java

Kelas anak:

- SegiEmpat.java
- SegiTiga.java
- Lingkaran.java

Program utama:

- TesPoligon.java



Polymorphism in Action : An Array of Shapes

```
abstract class Polygon {
    public abstract double area();
    public abstract String display();
}
class Rectangle extends Polygon {
    private double length, width;
    public Rectangle(double l, double w) { length = l; width = w; }
    @Override public double area() { return length * width; }
    @Override public String display() {
        return "Rectangle | area = " + area();
    }
}
// Circle and Triangle follow the same pattern

Polygon[] shapes = new Polygon[3];
shapes[0] = new Rectangle(17, 8);
shapes[1] = new Circle(10);
shapes[2] = new Triangle(6, 4);

for (Polygon p : shapes) {
    System.out.println(p.display());   // which display()?
}
```

Output

```
Rectangle | area = 136.0
Circle    | area = 314.16
Triangle  | area = 12.0
```

Add a Hexagon next month?

Write the Hexagon class.

The loop does NOT change, not one line.

KEY IDEA One loop, many shapes, no if-else chain.

The Object Class : toString() and equals()

- Every class extends Object, even without writing extends. Object is the root of every hierarchy.

```
class Point {
    private int x, y;
    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point other)) return false;
        return x == other.x && y == other.y;
    }
    @Override
    public int hashCode() { return Objects.hash(x, y); }
}
```

Without toString()

```
Point@3d8314f0
```

```
Point p1 = new Point(3, 4);
Point p2 = new Point(3, 4);
print(p1);
print(p1 == p2);
print(p1.equals(p2));
```

With the overrides

```
Point(3, 4)
false
true
```

⚠ **Override equals() → ALSO override hashCode().**

Group Project

The task

- Groups of at most 2 people.
- Choose a REAL problem , your own choice (library, clinic, parking, campus events…).
- Build a simple OOP program in Java.
- **Your design MUST include inheritance AND polymorphism, arising naturally from the problem.**

Presentation contents

- 1. The problem statement
- 2. Abstraction: classes + UML diagram showing inheritance
- **3. Attributes & methods : with REASONS for each access modifier**
- 4. A demonstration of polymorphism
- 5. Source code
- 6. Screenshots of the output

⚠ **Every member must be able to explain every line, and justify every access modifier.**

Thank You